

TraceMaker

A learning, GPU-assisted PCB autorouter and placer for KiCad
Design, method and benchmark results

Development report

2 October 2026

Abstract

TraceMaker reads KiCad 4–10 boards directly, routes them with an exact-geometry lattice router that learns from its own failures, and writes the tracks and vias back into the `.kicad_pcb` file without disturbing anything else. It also places components with an analytic placer checked by the router. Every result is judged by KiCad’s own design-rule checker (`kicad-cli pcb drc`). On Freerouting’s PCBench benchmark set TraceMaker routes 100% of the sampled tier-A boards cleanly, 65.0% of tier B, 56.7% of tier C and 50.0% of tier D. Freerouting 2.5.0-RC12’s published results on the same boards are 100%, 50.0%, 46.7% and 36.4%. No board in the final runs has a DRC error introduced by the router.

1 What TraceMaker does

A printed circuit board (PCB) starts as a set of footprints with copper pads and a netlist that says which pads must be connected. The router must draw copper tracks (and vias between layers) that make every connection while respecting the board’s design rules: minimum clearances between different nets, track widths, hole sizes, the board edge, solder-mask openings and more. A route is only useful if the board then passes the design-rule check (DRC) of the EDA tool the designer uses. TraceMaker therefore treats KiCad’s DRC as the judge.

Figure 1 shows the task on a real board from the benchmark set: a three-channel motor driver with fine-pitch QFN packages. The left panel shows the input, with the required connections drawn as straight airwires. The right panel shows TraceMaker’s result, which KiCad’s DRC reports as complete with no violations.

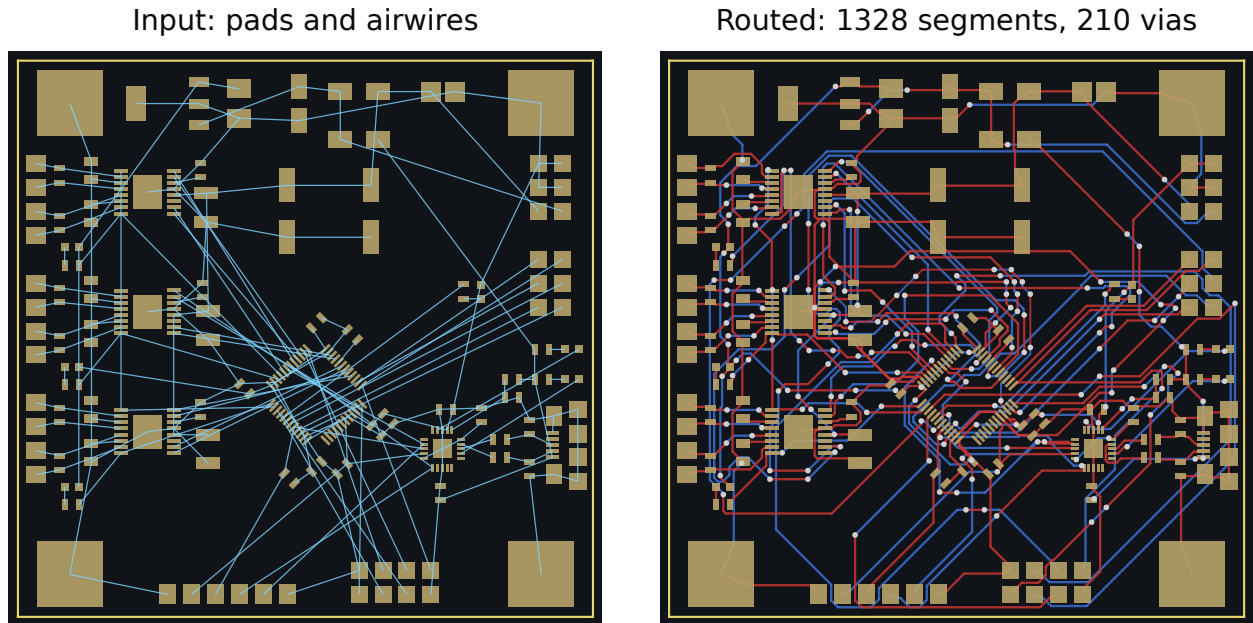


Figure 1: PCBench board `motor-3xdrv8833` (tier B, 195 connections). Left: input pads and airwires (a minimum spanning tree per net). Right: TraceMaker’s routing on the front (red) and back (blue) copper, with vias in grey. KiCad’s DRC: fully connected, no router-introduced violations.

2 Architecture

TraceMaker is about 12,500 lines of C++20 and CUDA, plus a 2,000-line TypeScript viewer and Python tooling. It runs on Ubuntu with an NVIDIA P100 and V100, but every GPU path has a CPU reference that gives identical results. Figure 2 shows the pipeline.

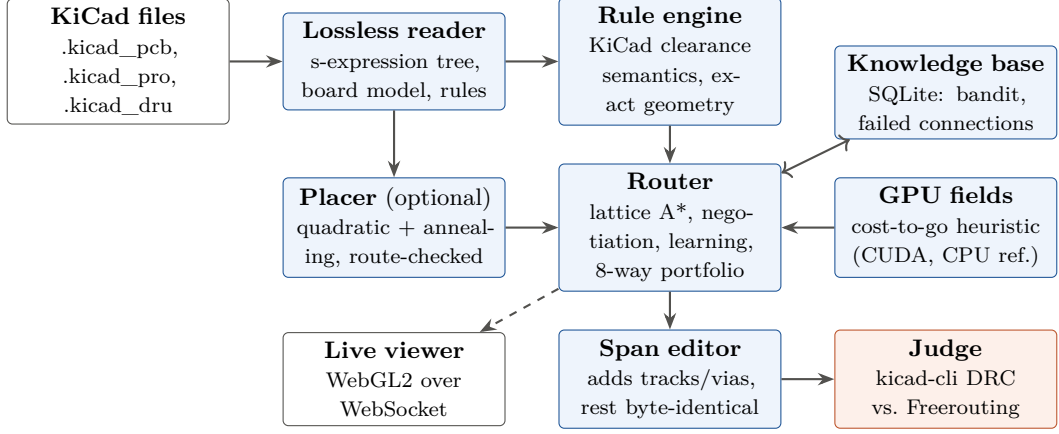


Figure 2: TraceMaker pipeline. The router never commits copper that the exact rule engine has not verified; the finished board is judged by KiCad’s own DRC.

Lossless KiCad I/O. The reader parses the board into an s-expression tree that remembers the byte span of every node. Edits (adding a track, moving a footprint) replace or append spans; everything else is written back byte for byte. All 3,538 fixture boards in the test set round-trip identically. The reader handles KiCad 4 to 10 formats, including legacy arcs, numbered and named nets, custom layer names and project rule files.

Exact geometry. All coordinates are 64-bit integer nanometres, as in KiCad. Shapes are “rounded cores”: a point, segment or polygon inflated by a radius, which represents round pads, tracks, vias and rounded rectangles exactly. Distance predicates use 128-bit integer arithmetic, so a clearance test never depends on floating-point rounding.

A KiCad-equivalent rule engine. Clearance is not a single number in KiCad. It is the maximum of the board minimum, the net-class clearances of both items, local pad and footprint overrides, diff-pair gaps and custom rules, with special cases for holes, the board edge and solder-mask openings. Each rule TraceMaker implements was checked against `kicad-cli` on real boards (Section 4). Examples found this way include: plated through-holes are on every copper layer whatever the file says; non-plated holes carry the board-edge clearance; a solder-mask opening exposes copper of its pad’s net on both sides; mask graphics grow by the board’s mask expansion; copper text is an obstacle.

3 How the router works

3.1 Search on an octilinear lattice

The board is covered by a lattice of points per copper layer. The pitch is a sixth of the smallest *track width + clearance* on the board (25–100 μm), so lattice tracks fit between fine-pitch pads. From each point a track may move in eight directions (0°, 45°, 90°, ...) or change layer through a via. A connection is found with A* search, which expands lattice points in order of

$$f(n) = g(n) + h(n), \quad (1)$$

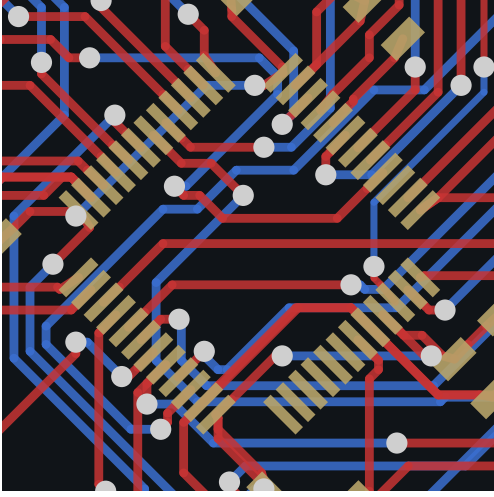
where $g(n)$ is the cost so far and $h(n)$ a lower bound on the remaining cost. The step cost is

$$c(u \rightarrow v) = \ell(u, v) + [\text{via}] c_{\text{via}} + [\text{bend}] c_{\text{bend}} + h_{\text{hist}}(v) + [\text{crosses other copper}] c_{\text{soft}}, \quad (2)$$

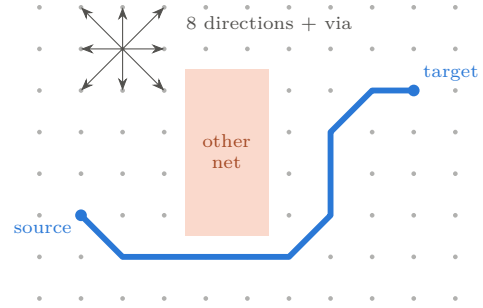
with track length ℓ , a via cost c_{via} (1mm of track by default), an optional bend penalty, the congestion history h_{hist} of Section 3.2 and, during negotiation only, a cost for crossing other connections’ copper. The heuristic h is the octile distance to the target or, on large search windows, an exact cost-to-go field computed on the GPU (Section 3.4).

Legality comes from the rule engine. Whether a lattice point is usable for a track of half-width $w/2$ on a given net is decided by the same rule engine as the DRC: the disk of radius $w/2$ around the point must keep the required clearance from every other net’s copper, from holes, the board edge, keepouts, solder-mask openings and copper text. Results for fixed copper are cached per net class and width; routed copper sits in a separate spatial index, and a count raster lets the search skip that index far from any routed copper.

Nothing is committed without an exact check. A lattice path is converted to straight segments (merging collinear steps) and every segment and via is verified exactly against the rule engine before it is written. If the check fails, the offending lattice points are learned as blocked for that net and the search is repeated. Short “escape stubs” off the lattice connect fine-pitch pads whose centres do not sit on lattice points.



(a) Close-up of fine-pitch QFN escapes on the motor-driver board (14 mm square). Pads in gold.



(b) Octilinear lattice search: each point has eight neighbours on its layer plus a via move; points within clearance of another net’s copper (shaded) are not usable.

Figure 3: Detailed routing on the lattice.

3.2 Learning from failure

The requirement was to “try the optimal first, store the failure information and use it to improve the next attempt”. TraceMaker does this at four levels.

1. **Within a search.** A failed search reports *why*: the source or target pad is boxed in (the open list ran out without reaching the window edge), the window was too small, the search budget ran out, or the exact check rejected the path. A boxed-in target is detected with a short reverse search from the target. The reason picks the next step instead of blindly retrying.
2. **Escalation ladder per connection.** Strict routing first (no crossing of other copper); then forced off-lattice escapes for pads with no lattice exit; then a neck-down to the board’s minimum track width; then negotiation.
3. **Negotiated rip-up (PathFinder).** In negotiation a path may cross other connections’ copper at a cost. The crossed connections are ripped up and queued again, and every contested lattice point accumulates a history cost

$$h_{\text{hist}}(v) \leftarrow h_{\text{hist}}(v) + 1 \quad \text{whenever } v \text{ is contested,} \quad (3)$$

so that later searches avoid congested places. This is the PathFinder idea of McMurchie and Ebeling (1995). “Nogoods” record attempts that failed in exactly the same surroundings, so they are not repeated, and the best legal state seen so far is always kept. When negotiation stalls, the router restarts from an empty board with the hardest connections first, keeping the history; once the planned restarts are used, further restarts clear the nogoods, halve the history and shuffle ties while budget remains.

4. **Across runs.** A SQLite knowledge base stores, per class of board, how well each router variant did and which connections failed on each board. A Thompson-sampling bandit chooses variants: each variant k has a $\text{Beta}(\alpha_k, \beta_k)$ posterior over its success rate, a sample θ_k is drawn for each, and the variants with the largest samples run. Connections that failed before are routed first next time.

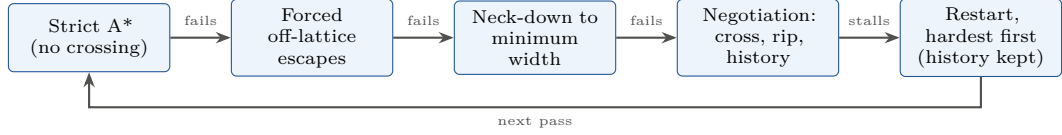


Figure 4: Escalation ladder for a connection that does not route. Each failure is classified, and the classification decides the next rung.

3.3 Parallel portfolio and determinism

Different strategies suit different boards: short connections first or long first, cheap or expensive vias, exact bend tracking or fast bends, and on large boards a lattice of double pitch. TraceMaker runs a portfolio of eight such variants in parallel threads and keeps the best result (most connections, then fewest vias, then shortest copper). When one variant routes everything the others stop.

Runs can be made fully deterministic with a work budget (`--work`, counted in A* expansions) instead of a time limit: random choices use a counter-based Philox generator keyed by (seed, stage, item), hash containers are never iterated in an output-affecting order, and costs are integers. Repeated runs then produce byte-identical boards, which the regression tests rely on.

3.4 GPU cost-to-go fields

On large search windows A* spends most of its time exploring around obstacles that a straight-line estimate does not see. Before such a search, a CUDA kernel computes the exact obstacle-aware cost to the target from every lattice point of the window (iterated GAMER-style sweeps over the grid). This field is a lower bound that is much tighter than the octile distance, so A* expands far fewer points. The field never prunes: where it claims “unreachable”, the router falls back to the octile bound, so the search stays complete. A CPU implementation produces bit-identical fields, and the tests compare them. Because the GPUs are shared with other jobs, a variant stops building fields if they take more than 30% of its wall-clock time.

3.5 Component placement

Placement (`tracemaker-place`) works in stages: a quadratic wirelength minimisation with the bound-to-bound net model, solved by conjugate gradients; SimPL-style spreading to remove overlap; rotation choice; legalisation to the nearest legal position (courtyards, board outline, copper and solder-mask clearances); and parallel simulated annealing on half-perimeter wirelength

$$\text{HPWL} = \sum_{\text{nets } e} \left(\max_{p \in e} x_p - \min_{p \in e} x_p + \max_{p \in e} y_p - \min_{p \in e} y_p \right) \quad (4)$$

plus a penalty per airwire crossing. A linear-programming lower bound on HPWL, solved as a min-cost flow, is reported alongside so the quality of each placement can be judged.

Shorter wiring does not automatically mean a board routes better: denser packing can block escape routes. The placer therefore puts the router in the loop. With `--route-check`, the input placement and the new one are routed with the same deterministic budget, and the new placement is kept only if it leaves no more connections unrouted. `--mode auto` does this for both the refine and the full mode and keeps the best of the three.

3.6 Live view and KiCad integration

`tracemaker route --view` streams the routing as it happens to a WebGL2 viewer in the browser: tracks appearing and being ripped up, the A* frontier, failures and the remaining airwires. A KiCad 10 action plugin routes the open board through KiCad’s IPC API and adds the result in one undoable commit.

4 How it was tested

- **Unit and integration tests** (43, run with `ctest`): exact unit conversion, Philox known-answer vectors, GPU-equals-CPU for the random generator and cost-to-go fields, s-expression parsing and byte-exact round trips, the board reader against KiCad’s own model, rule reading, geometry predicates, placement legality and lower bounds, and the viewer server.

- **DRC parity.** TraceMaker’s own DRC is compared with `kicad-cli` on the KiCad demo boards; 11 of 16 match exactly on the 15 routing-relevant violation types, and these 11 are a regression test. The router uses the same rule engine, so parity problems show up as router errors and were fixed at the source.
- **Benchmark harness.** `bench/run.py` samples boards from Freerouting’s PCBench set by Freerouting’s own difficulty tiers, routes each with a snapshot of the binary (120s, eight variants), and runs `kicad-cli pcb drc` on the result. A board is *clean* when it is fully connected and KiCad reports no violation involving a track, via or arc that was not already present in the unrouted board. The result is compared with Freerouting’s published per-board results for the same files.

5 Results

5.1 Clean pass by tier

Figure 5 and Table 1 give the final run (`final18`). TraceMaker matches Freerouting 2.5.0-RC12 on the routine tier-A boards and is ahead on the harder tiers. No board has a router-introduced DRC error.

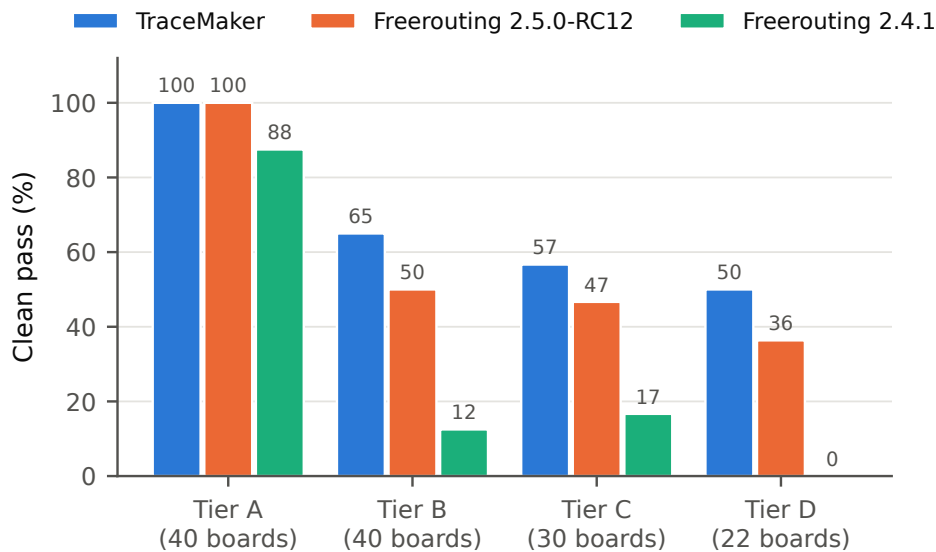


Figure 5: Share of boards routed clean (fully connected, no router-introduced KiCad DRC errors), 120s per board. Freerouting figures are its published results on the same fixture files.

Tier	Boards	TraceMaker	FR 2.5.0-RC12	FR 2.4.1	Router errors
A (routine 2-layer)	40	100%	100%	87.5%	0
B (2-4 layer)	40	65.0%	50.0%	12.5%	0
C (complex)	30	56.7%	46.7%	16.7%	0
D (hardest)	22	50.0%	36.4%	0.0%	0

Table 1: Final benchmark run. Samples: 40 of 453 tier-A boards, 40 of 560 tier-B, 30 of 122 tier-C and all 22 tier-D boards (fixed seed). FR: Freerouting’s published results. Router errors: boards with router-introduced DRC errors.

Figure 6 compares the two routers board by board. On every tier there are more boards that only TraceMaker routes clean than boards that only Freerouting does.

5.2 Progress during development

Figure 7 tracks every full benchmark run during the development session. The first router (plain lattice A*) routed 70% of tier A clean. Fixing rule semantics found against KiCad, the learning mechanisms and the escalation ladder took tier A to 100% within about two and a half hours; the later gains on tiers B–D came from boxed-in detection at both ends of a connection, pad-approach fixes for fine-pitch parts, the coarse-lattice variants for large boards, extra restarts and the large-board setup fix.

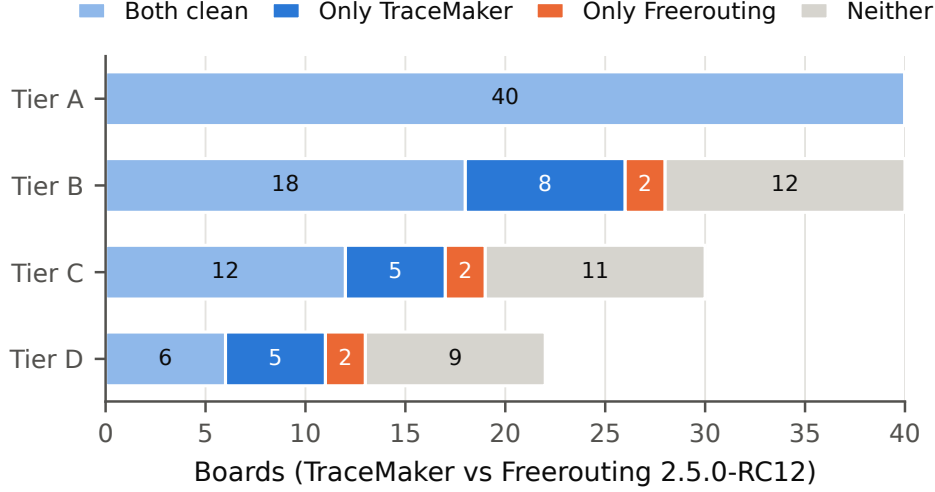


Figure 6: Board-by-board comparison with Freerouting 2.5.0-RC12 in the final run.

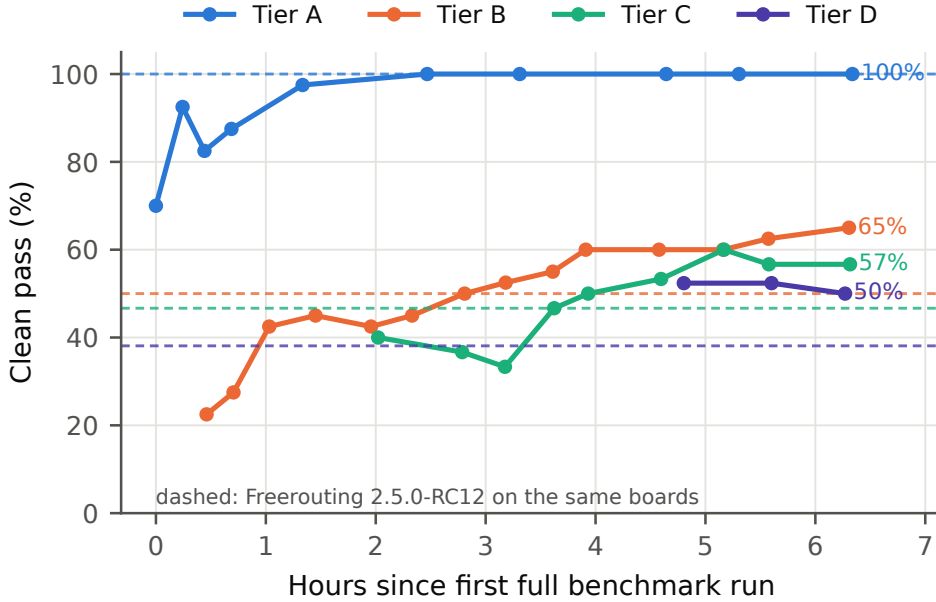


Figure 7: Clean pass of every full benchmark run (20 or more boards) over the development session. Dashed lines: Freerouting 2.5.0-RC12 on the same boards. Small run-to-run differences come from the wall-clock limit.

5.3 Completion and speed

Clean pass is all-or-nothing. Figure 8a shows how close the remaining boards are: most boards that are not clean miss only a few connections, and mean completion is above 95% on every tier.

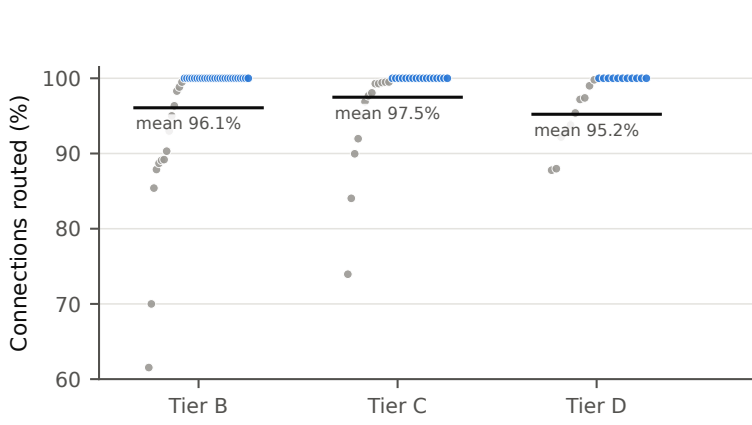
On tier A the median TraceMaker routing time is 0.9s per board (1.7s including file input and output), and every board finished faster than Freerouting’s published time (Figure 8b).

5.4 Routed boards

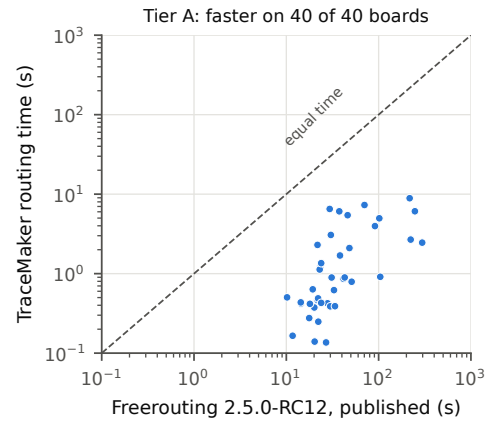
Figure 9 shows a selection of routed boards from the final run, rendered from the output files. The four-layer high-voltage switching board (604 connections) uses the inner layers (yellow and green); **draco** and **EUC-VESC** are boards that TraceMaker routes clean and Freerouting does not.

5.5 Placement

The placer was evaluated on 23 PCBench boards in `--mode auto` with a router check of three million A* expansions (Figure 10). It kept a new placement on 18 boards and the human placement on five. Total HPWL fell from 17,464 mm to 13,190 mm (−24.5%, median −17% per board), unrouted connections fell from 61 to



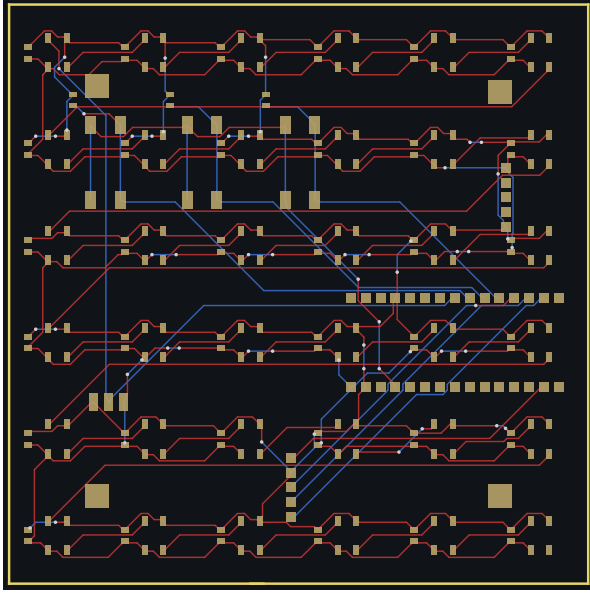
(a) Share of connections routed per board. Blue: fully routed.



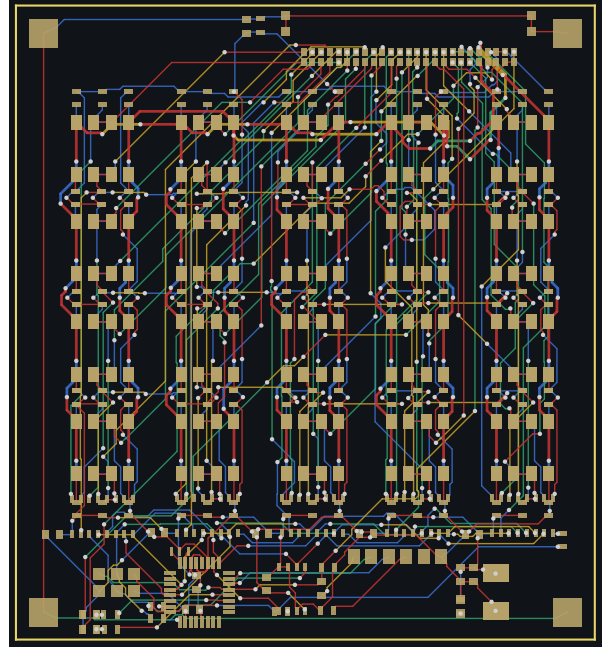
(b) Tier-A routing time against Freerouting's published time.

Figure 8: Completion on the harder tiers and speed on tier A. Freerouting's times were measured on its own hardware, so the comparison is indicative only; TraceMaker used eight threads of a 56-core server.

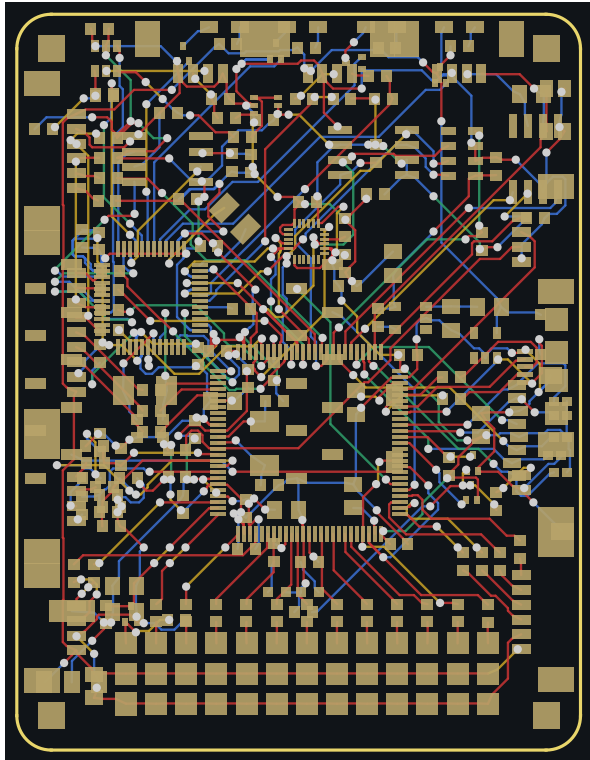
59, and no board got worse. Without the router check, placement alone cut wirelength by a similar amount but left 13 boards fully routed instead of 15: denser packing can block routing, which is why the check exists.



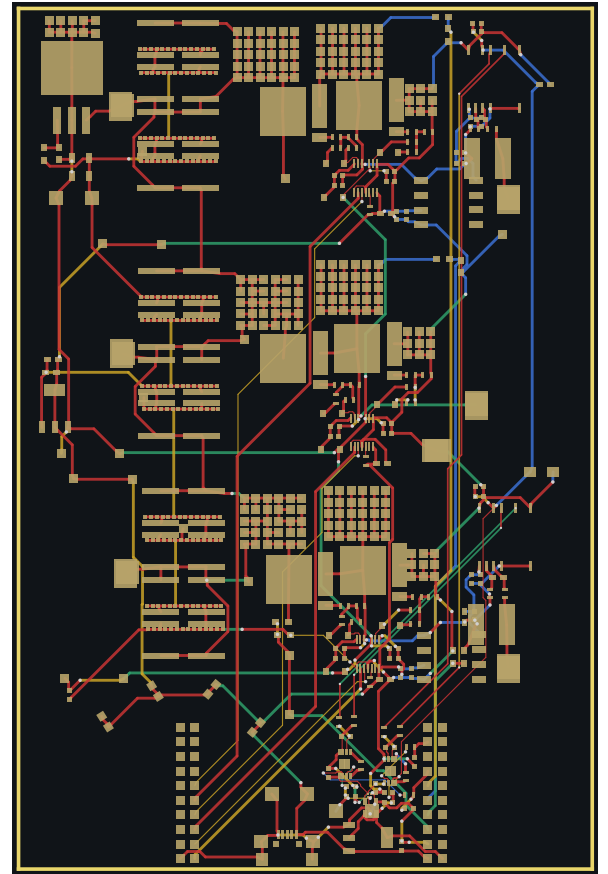
(a) Hangul-Clock (tier A): 209 connections, 6 s.



(b) 40-channel-hv-switching-board (tier C, 4 layers): 604 connections.



(c) draco (tier C): 418 connections; clean only with TraceMaker.



(d) EUC-VESC (tier D): 760 connections; clean only with TraceMaker.

Figure 9: Boards routed by TraceMaker in the final benchmark run, each fully connected with no router-introduced KiCad DRC errors. Red: front copper; blue: back copper; yellow and green: inner layers; gold: pads; grey: vias.

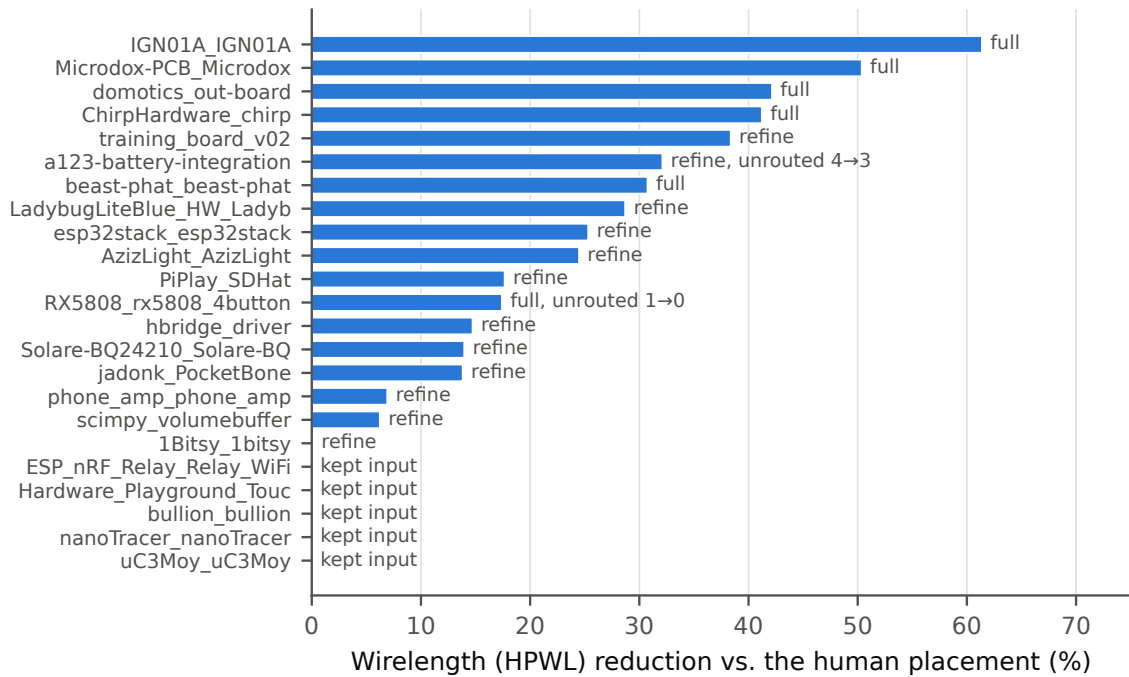


Figure 10: Wirelength reduction of the placement kept by `tracemaker-place --mode auto`, per board. “Kept input”: the router check kept the human placement. Labels give the mode kept and any change in unrouted connections.

6 Limitations and next steps

- **Large boards are time-limited.** On the biggest boards the router finishes only one or two negotiation passes in 120s; an A* expansion costs about 430 ns. Global routing corridors, a coarse-to-fine lattice and parallelism inside one route are the next steps.
- **Edge-connector fingers under solder-mask openings** are left unrouted rather than routed with mask bridges. On the PCIe adapter board KiCad flags any track leaving the fingers, and the human-routed original has 16 such bridges itself. Freerouting’s published “clean” counts only clearance violations and unrouted connections, so on these boards TraceMaker is judged more strictly.
- **Copper text** is modelled by generous estimated boxes rather than exact glyph outlines.
- **Placement** does not yet have a routability term inside the annealer, side flipping or decoupling-capacitor grouping.
- **Not yet built:** schematic-to-board flow, escape planning for large BGAs, differential-pair and length tuning.

A Reproducing the results

```
cmake --preset release && cmake --build --preset release && ctest --preset release
scripts/fetch_fixtures.sh pcbench
python3 bench/run.py --tier B --limit 40 --time 120 --jobs 2 --threads 8 --name my-run
build/release/src/app/tracemaker route board.kicad_pcb -o routed.kicad_pcb --time 120 --view
build/release/src/place/tracemaker-place in.kicad_pcb -o out.kicad_pcb \
    --mode auto --route-check 3000000
build/report-venv/bin/python report/make_figures.py # figures for this report
```